Artificial World — Paper Fundacional · 2026-03-08 · Repositorio: `artificial-word` · Autor: Cosigein SL

Artificial World: Arquitectura de un Motor de Civilizaciones Emergentes con Agentes Autónomos Basados en Utilidad, Memoria Persistente y Auditoría Trazable

Autor: Cosigein SL

Fecha: 2026-03-08

Repositorio: `artificial-word` — github.com/artificial-word

Versión: 1.0 — Paper fundacional

Abstract

Presentamos **Artificial World**, un sistema de simulación de vida artificial 2D orientado a la generación de civilizaciones emergentes con memoria persistente, héroes fundadores, refugios como unidades de organización social, y comunidades con tensiones internas. El sistema implementa dos capas técnicas diferenciadas: un motor Python con función de utilidad multidimensional (13 tipos de acción, 9 modificadores, ruido determinista por semilla), y una capa web fullstack en Node.js/React con siete arquetipos de civilización (*CivilizationSeed*) y un sistema de refugios persistentes (*HeroRefuge*). Ambas capas son deliberadamente independientes, compartiendo un modelo conceptual común pero no una integración técnica en tiempo real, lo que constituye una decisión arquitectónica explícita documentada y defendida. El sistema incorpora además un mecanismo de auditoría independiente con agentes dockerizados en red aislada (*Sistema Chess*), un modo de simulación especular sin efectos secundarios (*Modo Sombra*), un sistema de vigilancia activa (*Watchdog*) y una infraestructura documental que genera crónicas verificables en JSON, Markdown y PDF desde el propio código. Los resultados de la sesión canónica (semilla 42, 200 ticks, 8 agentes) revelan comportamientos emergentes no programados: cohesión grupal bajo escasez severa (62.4% de acciones `SEGUIR`), agotamiento zonal en el cuadrante oeste, y divergencia individual exitosa como estrategia no codificada. El paper argumenta que la trazabilidad radical —cada decisión de agente es reproducible, cada claim de sistema es verificable contra el código— es una propiedad de diseño de primer orden, no una característica opcional.

Palabras clave: vida artificial, agentes autónomos, función de utilidad, civilizaciones emergentes, simulación 2D, memoria social, modo sombra, auditoría de código, trazabilidad, comportamiento emergente.

1. Introducción

1.1 El problema de la afirmación sin evidencia

El campo de la inteligencia artificial atraviesa una paradoja: nunca ha habido más capacidad de generar texto convincente sobre sistemas, y nunca ha sido más difícil distinguir qué parte de un sistema existe realmente de qué parte existe solo como afirmación. Esta paradoja no es únicamente un problema de marketing: afecta directamente a la arquitectura de los sistemas. Un sistema diseñado para *parecer* capaz tiende a optimizar la superficie de presentación en detrimento de la profundidad verificable.

Artificial World parte de la premisa contraria: **la trazabilidad radical es una propiedad de diseño de primer orden**. Cada comportamiento de agente debe ser reproducible mediante semilla fija. Cada claim de capacidad debe apuntar a una línea de código. Cada crónica generada debe ser verificable contra el estado del sistema en el tick correspondiente.

1.2 Motivación y contexto

Los sistemas clásicos de vida artificial (Conway, 1970; Reynolds, 1987; Langton, 1989) priorizan la simplicidad de las reglas locales sobre la riqueza del estado interno de los agentes. Los sistemas más complejos (Creatures, Dwarf Fortress, RimWorld) enriquecen el estado interno pero tienden a oscurecer los mecanismos de decisión bajo capas de abstracción que dificultan la auditoría.

Artificial World busca un punto de equilibrio diferente: **estado interno rico y decisión completamente transparente**. La función de utilidad que gobierna cada decisión de cada agente en cada tick es pública, parametrizada y reproducible. No hay componentes estocásticos no acotados: el ruido es determinista por semilla.

Adicionalmente, el sistema propone una separación explícita entre:

- la **verdad sistémica** (lo que existe, lo que está implementado, lo que los tests verifican)
- la **tesis de producto** (lo que el sistema se propone demostrar)
- la **visión futura** (lo que el sistema podría llegar a ser con trabajo adicional)

Esta separación, documentada en los metadocumentos del repositorio (`DOCUMENTO_UNICO.md`, `VISION_CIVILIZACIONES_VIVAS.md`, `ESTRATEGIA_PRODUCTO.md`), es en sí misma una contribución metodológica: un modelo de honestidad técnica aplicado a un proyecto de software activo.

1.3 Contribuciones principales

Este trabajo documenta las siguientes contribuciones:

1. **Arquitectura dual de motor de simulación**: separación explícita entre motor de alta fidelidad (Python/pygame) y capa de demostración web (Node.js/React), con modelo conceptual compartido y sin integración técnica forzada.
2. **Función de utilidad multidimensional con 9 modificadores**: incluyendo estado fisiológico, rasgos de personalidad, memoria espacial y social, relaciones inter-agente, directivas externas y capa de autonomía de emergencia.

3. **Sistema de civilizaciones con semilla arquetipal**: siete arquetipos (*CivilizationSeed*) que condicionan valores, tensiones, héroe probable y tono del mundo desde el momento fundacional.

4. **Modo Sombra**: simulación especular que evalúa contrafácticos sin modificar el estado observable del mundo principal.

5. **Sistema Chess**: auditoría independiente con seis agentes dockerizados especializados en red aislada con montaje read-only del repositorio.

6. **Infraestructura de crónica automática**: generación de historiales verificables en JSON, Markdown y PDF desde el runtime de simulación.

7. **Modelo de trazabilidad radical**: como propiedad de diseño, no como característica opcional.

---

2. Trabajo Relacionado

2.1 Sistemas de vida artificial basados en agentes

La tradición de la vida artificial (Alife) se divide en dos familias principales. Los **sistemas de reglas locales** (autómatas celulares de Conway, 1970; Boids de Reynolds, 1987) logran comportamiento emergente complejo a partir de reglas simples por agente. Los **sistemas de agentes con estado interno** (Creatures, The Sims, Dwarf Fortress) priorizan la riqueza del estado individual sobre la simplicidad de las reglas.

Artificial World pertenece a la segunda familia, pero con una distinción importante respecto a los sistemas existentes: la **decisión por utilidad es completamente transparente y reproducible**. En Dwarf Fortress (Adams, 2006), el sistema de needs y thoughts de los enanos es verificable a nivel de UI pero opaco a nivel de código. En Artificial World, la función de puntuación es pública, parametrizable y reproducible mediante semilla.

2.2 Función de utilidad en sistemas multiagente

La teoría de la utilidad esperada (Von Neumann & Morgenstern, 1944) ha sido adaptada extensamente para sistemas multiagente. La arquitectura BDI (Belief-Desire-Intention) de Rao & Georgeff (1991) propone separar creencias, deseos e intenciones como componentes del razonamiento deliberativo. Artificial World simplifica esta separación reemplazando la selección de intenciones por una **función de puntuación continua** que permite graduación natural entre opciones competidoras, con fallback garantizado a `DESCANSAR` cuando ninguna acción es viable.

La ventaja de esta aproximación frente a la arquitectura BDI completa es la reducción de complejidad de implementación sin sacrificar la capacidad de expresar preferencias graduadas. La desventaja es la mayor dificultad para modelar compromisos de largo plazo (goals) que persistan sobre múltiples ticks.

2.3 Sistemas de refugio y organización social emergente

RimWorld (Sylvester, 2013) trata la colonia como unidad de organización, pero las relaciones entre colonos son diádicas y el concepto de "colonia" no tiene estado propio más allá de la suma de sus miembros. Dwarf Fortress tiene una noción de fortaleza más rica, pero los sistemas de legado, memoria y diplomacia son difíciles de auditar externamente.

Artificial World formaliza el **refugio como entidad de primer orden** con estado propio independiente de sus ocupantes: `nombre`, `recursos`, `seguridad`, `moral`, `amenazas`, `memoria_local`, `fase_de_crecimiento`. Esta formalización es importante porque permite que la historia del refugio persista incluso si todos sus ocupantes originales mueren o migran.

2.4 Sistemas de memoria en agentes autónomos

La memoria espacial en agentes simulados ha sido tratada principalmente como mapa de exploración (sistemas de niebla de guerra). Artificial World extiende este concepto con **memoria social**: cada agente mantiene vectores de `confianza`, `hostilidad` y `miedo` por par de agentes, que evolucionan con cada interacción y condicionan directamente la función de utilidad en los ticks posteriores.

Esta memoria social retroalimenta el sistema de acciones sociales: un agente que recuerda haber sido robado por otro incrementa su `hostilidad` y `miedo` hacia ese agente, reduciendo la probabilidad de acciones de `COMPARTIR` y `SEGUIR` con él en el futuro.

---

3. Arquitectura del Sistema

3.1 Capas del sistema

El sistema se compone de tres capas independientes con un modelo conceptual compartido:

```
┌──────────────────────────────────────────────────────────┐
│              MODELO CONCEPTUAL COMPARTIDO                 │
│  World · CivilizationSeed · Refuge · Hero · Community     │
│  MemoryEntry · HistoricalRecord · Territory               │
└──────────────────┬──────────────────────┬─────────────────┘
                   │                      │
┌──────────────────▼──────┐  ┌────────────▼──────────────┐
│   MOTOR PYTHON (REAL)   │  │   CAPA WEB (DEMO FUNCIONAL) │
│                         │  │                            │
│  principal.py           │  │  backend/src/ (Node/Express)│
│  nucleo/simulacion.py   │  │  frontend/src/ (React/Vite) │
│  agentes/               │  │  HeroRefuge + 7 semillas    │
│  acciones/ (13 tipos)   │  │  civilizationSeeds.js       │
│  sistemas/ (persistencia│  │  aiCore.js (Ollama)         │
│    Modo Sombra, watchdog│  │                            │
│    competencia)         │  └────────────────────────────┘
│  mundo/ (grid 60×60)    │
│  SQLite (persisted)     │  ┌────────────────────────────┐
│  68+ tests              │  │  SISTEMA CHESS (AUDITORÍA)  │
│                         │  │  6 agentes dockerizados     │
└─────────────────────────┘  │  red aislada (read-only)    │
                             │  REPORTE_CHESS_1.md         │
                             └────────────────────────────┘
```

La separación entre motor Python y capa web **no es un error de diseño sino una decisión arquitectónica explícita**, documentada en `docs/DECISION_PUENTE_PYTHON_JS.md`. Las razones son:

1. Cada motor está optimizado para su contexto (alta fidelidad vs. demostración web)
2. No existe necesidad operativa de integración en tiempo real a corto plazo
3. El coste de implementar el puente (API Python o WASM) supera el beneficio en la fase actual
4. La narrativa clara de "dos motores, dos propósitos" es más honesta que una integración superficial

3.2 Motor Python: ciclo de tick

El orquestador principal (`nucleo/simulacion.py` , ~848 líneas) inicializa y coordina doce subsistemas:

Subsistema	Clase	Función
Ticks	`GestorTicks`	Control de tiempo y velocidad
Eventos	`BusEventos`	Publicación/suscripción de eventos entre sistemas
Logs	`SistemaLogs`	Registro estructurado de la sesión
Métricas	`SistemaMetricas`	Conteo de acciones y estados
Regeneración	`SistemaRegeneracion`	Reposición de recursos en el mapa
Persistencia	`SistemaPersistencia`	Guardar/cargar en SQLite
Watchdog	`SistemaWatchdog`	Alertas de condiciones anómalas
Reporte	`SistemaReporte`	Generación de crónica fundacional
Competencia	`SistemaModoCompetencia`	Modo multi-jugador por turnos
Modo Sombra	`GestorModoSombra`	Simulación especular
Renderizador	`Renderizador`	Representación visual pygame

El ciclo por tick, por entidad, es:

```
actualizar_estado_interno()          ← fisiología: hambre++, energía--
percibir_entorno()                   ← percepción local + amenaza_local
actualizar_memoria()                 ← registra recursos/refugios visibles
actualizar_directivas()              ← expira directivas caducadas
ContextoDecision(tick, mapa, ...)    ← construye contexto de decisión
decidir_accion() → MotorDecision     ← función de utilidad → acción ganadora
ejecutar_accion()                    ← aplica efectos en el mundo
```

3.3 Función de utilidad: formulación completa

La puntuación de una acción \(a\) para el agente \(i\) en el tick \(t\) es:

$$U(a, i, t) = U_{\text{base}}(a) + \sum_{k=1}^{9} \delta_k(a, i, t) + \epsilon(i, t)$$

donde:

- $U_{\text{base}}(a)$ es la utilidad intrínseca de la acción (tabla fija por tipo, ver Tabla 1)
- δ_k son los nueve modificadores contextuales (ver Tabla 2)

- $\epsilon(i,t) \sim \mathcal{U}(-0.08, 0.08)$ es ruido determinista: `random.Random(entidad.id * 1000 + tick).uniform(-0.08, 0.08)`

Tabla 1. Utilidades base por tipo de acción

| Acción | U_{base} | Condición de viabilidad |

|---|---|---|

| `COMER` | 0.65 | Comida en inventario > 0 |

| `RECOGER_COMIDA` | 0.55 | Celda actual tiene recurso COMIDA |

| `EXPLORAR` | 0.50 | Energía > 0.20 |

| `HUIR` | 0.40 | Riesgo percibido > 0.3 |

| `EVITAR` | 0.25 | Riesgo percibido > 0.3 |

| `DESCANSAR` | 0.30 | (fallback garantizado) |

| `MOVER` | 0.30 | Posición vecina disponible |

| `RECOGER_MATERIAL` | 0.30 | Celda actual tiene recurso MATERIAL |

| `IR_REFUGIO` | 0.25 | Refugio adyacente + (energía < 0.45 OR riesgo > 0.5) |

| `COMPARTIR` | 0.20 | Inventario ≥ 3 OR (cooperativo + inventario ≥ 1 + hambre < 0.5) |

| `ROBAR` | 0.20 | Oportunista/agresivo + hambre ≥ 0.75 + sin comida + energía > 0.30 |

| `SEGUIR` | 0.20 | Cooperativo/neutral + entidades visibles cercanas |

| `ATACAR` | 0.18 (var.) | Condición contextual agresiva |

> `COMER` *tiene la máxima prioridad base (0.65) porque solo es candidato cuando hay comida en inventario — condición de alta certeza de éxito.*

Tabla 2. Los nueve modificadores de la función de utilidad

| # | Modificador | Variable fuente | Efecto |

|---|---|---|---|

| 1 | `mod_hambre` | `estado.hambre` | Escala utilidad de COMER/RECOGER con la hambre actual |

| 2 | `mod_energia` | `estado.energia` | Aumenta utilidad de DESCANSAR cuando energía < 0.6 |

| 3 | `mod_riesgo` | `estado.riesgo_percibido` | Aumenta HUIR/EVITAR cuando riesgo > 0.3 |

| 4 | `mod_rasgo` | `rasgo_principal` | Ajusta preferencias según arquetipo conductual |

| 5 | `mod_anti_oscilacion` | `posicion_anterior` | Penaliza −0.40 si destino MOVER = celda anterior |

| 6 | `mod_memoria` | `memoria.recursos_recientes` | +0.15–0.25 hacia comida visible/recordada; +0.10 hacia refugio |

| 7 | `mod_relaciones` | `relaciones.obtener_relacion()` | Confianza → COMPARTIR/SEGUIR; hostilidad → HUIR/EVITAR |

| 8 | `mod_directivas` | `contexto.directivas_activas` | Órdenes externas con intensidad variable |

| 9 | `mod_autonomia` | múltiple | Override de emergencia: supervivencia anula directivas |

La capa de autonomía como override de emergencia:

La capa `mod_autonomia` actúa como corrector de último recurso independiente de cualquier directiva:

```
if hambre > 0.85:      penaliza EXPLORAR y MOVER_LIBRE
if energia < 0.15:     penaliza MOVER y ROBAR
if riesgo > 0.80:      bonifica HUIR sobre cualquier directiva activa
```

Esta capa garantiza que ningún agente pueda ser enviado a su muerte por una directiva mal calibrada. Es un mecanismo de seguridad intrínseco al diseño.

Directivas externas (modo investigador):

El modificador `mod_directivas` permite al operador exterior emitir instrucciones a agentes individuales con intensidades variables:

| Directiva | Efecto en utilidad |

|---|---|

| `PRIORIZAR_SUPERVIVENCIA` | +0.85 a acciones de subsistencia |

| `IR_A_POSICION` | +1.50 a la acción de movimiento hacia destino |

| `QUEDARSE_AQUI` | −2.00 a todas las acciones de desplazamiento |

| `EXPLORAR_ZONA` | Bonus proporcional a exploración en área objetivo |

3.4 Rasgos conductuales y arquetipo de agente

Cada agente tiene un rasgo principal que modifica su función de utilidad:

Agentes sociales (tipo `EntidadSocial`):

| Rasgo | Bonus principal | Penalización principal |

|---|---|---|

| `COOPERATIVO` | COMPARTIR +0.25, SEGUIR leve | ROBAR −0.25 |

| `NEUTRAL` | — | — |

| `AGRESIVO` | ROBAR +0.30 | COMPARTIR −0.20, HUIR −0.10 |

| `EXPLORADOR` | MOVER +0.35, EXPLORAR +0.35 | SEGUIR −0.05 |

| `OPORTUNISTA` | ROBAR +0.15, RECOGER_MATERIAL +0.08 | leve COMPARTIR |

Agentes tipo `EntidadGato` (5 rasgos felinos diferenciados):

| Rasgo | Tendencia |

|---|---|

| `CURIOSO` | Bonus explorar/mover (+0.35), bonus seguir leve |

| `APEGADO` | Penaliza mover/explorar (−0.05), bonus seguir (+0.25) |

| `INDEPENDIENTE` | Bonus mover/explorar (+0.20), penaliza seguir (−0.25) |

| `TERRITORIAL` | Bonus ir_refugio (+0.15), penaliza mover/explorar |

| `OPORTUNISTA` | Bonus mover/explorar/recoger_comida, bonus seguir |

3.5 Memoria de agente: estructura y uso

Cada agente mantiene dos estructuras de memoria persistentes entre ticks:

Memoria espacial: registro de posiciones de recursos (comida, material) y refugios observados recientemente. El modificador `mod_memoria` usa esta estructura para generar bonus direccionales: si un agente recuerda una celda con comida, recibe un bonus de +0.15 a +0.25 en acciones de movimiento hacia esa celda, proporcional a su nivel de hambre actual.

Memoria social: para cada par de agentes (i, j), se mantiene un vector tridimensional $(\text{confianza}_{ij}, \text{hostilidad}_{ij}, \text{miedo}_{ij}) \in [0, 1]^3$. Este vector evoluciona con cada interacción:

| Evento | Efecto en relaciones |

|---|---|

| `COMPARTIO` | Donante: confianza +1.2; Receptor: confianza +1.5 |

| `ROBO` | Víctima: hostilidad +0.4, confianza −0.3 |

| `ATAQUE_EJECUTADO` | Objetivo: hostilidad +0.4, confianza −0.3 |

| `SIGUIO` | Leve incremento de confianza bilateral |

3.6 Modo Sombra: simulación especular

El Modo Sombra (`sistemas/gestor_modo_sombra.py`) implementa una simulación paralela que replica el estado del mundo principal pero acepta intervenciones sin modificar el mundo observable. Permite:

1. **Evaluación de contrafácticos**: ¿qué habría ocurrido si el agente X hubiera tomado la decisión Y en el tick T?
2. **Control manual**: el operador puede asumir el control de un agente específico con entrada WASD en tiempo real, la IA cede el turno para ese agente
3. **Modo investigador**: directivas experimentales sin contaminar la simulación principal

El Modo Sombra no se activa en la sesión canónica estándar pero está completamente verificado mediante suite de tests propia.

3.7 Sistema Watchdog: vigilancia activa

El subsistema `SistemaWatchdog` monitoriza condiciones anómalas en tiempo real y emite alertas estructuradas:

```
{
  "nivel": "CRITICAL",
  "codigo": "HAMBRE_CRITICA_SIN_RESPUESTA",
  "tick": 189,
  "entidad_id": 2,
  "entidad_nombre": "Bruno",
  "descripcion": "Hambre >= 0.9 durante 8 ticks consecutivos sin acción COMER"
}
```

Las alertas forman parte de la crónica fundacional y son verificables contra el log de acciones del mismo período.

---

4. Capa Web: Semillas, Refugios y Civilizaciones

4.1 Los siete arquetipos de civilización

La capa web implementa siete `CivilizationSeed` con estado fundacional completo:

| ID | Arquetipo | Valores centrales | Tensiones internas | Héroe probable |

|---|---|---|---|---|

| `frontier-tribe` | Survivalist | Adaptación, cooperación, resistencia | Clima hostil, escasez, frontera inestable | Explorador/Guardián |

| `technocrat-refuge` | Planner | Orden, eficiencia, conocimiento | Rigidez, elitismo, dependencia energética | Ingeniero/Estratega |

| `spiritual-community` | Ritual | Fe, vínculo, memoria | Dogma, miedo al cambio, presagios | Guardián/Profeta |

| `warrior-kingdom` | Militant | Honor, fuerza, jerarquía | Expansión, rivalidad, coste humano | Campeón/Conquistador |

| `merchant-city` | Mercantile | Intercambio, movilidad, acuerdos | Desigualdad, corrupción, dependencia de rutas | Negociador/Explorador |

| `paranoid-colony` | Fortress | Seguridad, vigilancia, autarquía | Sospecha, aislamiento, fractura interna | Vigía/Ingeniero |

| `decadent-empire` | Legacy | Linaje, poder, apariencia | Declive, intriga, sobrecoste | Heredero/Diplomático |

Cada semilla genera un estado fundacional completo vía `createFoundingWorldState()`:

```
{
  name: "Refugio Fundador",
  resources: { food: 60, shelter: 75, security: 55, morale: 58 },
  foundingHero: { name, role: "Constructor de Mundos", archetype },
  community: { name, culture, tensions, cohesion: 0.72 },
  memory: [{ type: "founding", summary: "...", significance: 10 }],
```

```
  territory: { influenceRadius: 1, routes: [] }
}
```

4.2 HeroRefuge: el sistema de persistencia de mundos

`HeroRefuge` gestiona hasta 256 mundos vivos persistentes en SQLite, con:

- **Companion IA** (`PersonalAgent`): memoria circular de 100 entradas, rasgos estables (`loyalty=1.0, curiosity=0.8`), fallback determinista si Ollama no está disponible
- **Mundo artificial** (`ArtificialWorld`): recursos con degradación estocástica, 5% probabilidad de evento aleatorio por tick (7 tipos: discovery, threat, birth, death, conflict, alliance, migration)
- **Persistencia debounced**: escritura a SQLite tras 2s de inactividad post-modificación

4.3 IA local: diseño sin dependencias de pago

El módulo `aiCore.js` utiliza Ollama como proveedor local. Su diseño incluye:

1. **Fallback determinista**: si Ollama no responde, genera texto estructurado desde datos locales sin inventar
2. **Tracing completo**: cada operación registra `{operation, provider, model, durationMs, success, fallback, error}`
3. **Memoria estructurada**: las operaciones `analyzeTestFailure` y `analyzeSession` consultan claves de memoria específicas antes de llamar al LLM

Esta arquitectura garantiza que el sistema funciona sin conexión a internet y sin coste por token en el núcleo.

---

5. Sistema Chess: Auditoría Independiente

5.1 Filosofía del sistema

El Sistema Chess resuelve un problema fundamental en proyectos donde el mismo equipo construye y audita: **los creadores no pueden auditarse a sí mismos con objetividad**. La solución es un conjunto de agentes completamente separados, sin acceso de escritura, que analizan el mismo repositorio desde fuera.

Principios de diseño:

- Los agentes montan el repositorio en modo **READ-ONLY** (`:ro`)
- Cada ejecución es desde cero (sin estado entre ejecuciones)
- Los agentes no pueden comunicarse con producción ni con el entorno de tests
- Cada agente tiene una especialidad y un dominio de preguntas acotado

5.2 Los seis agentes auditores

| Agente | Especialidad | Dominio de detección |

|---|---|---|

| `agent-docs` | Documentación | Claims prohibidos, enlaces rotos, inconsistencias README |

| `agent-backend` | API y seguridad | `console.log`, URLs hardcodeadas, `catch` vacíos |

| `agent-frontend` | React | Componentes >300 líneas, imports incorrectos, sin `alt` |

| `agent-bd` | Base de datos | SQL injection patterns, `SELECT *`, sin `IF NOT EXISTS` |

| `agent-tests` | Cobertura | Tests sin assertions, ratio cobertura <30%, TODOs |

| `agent-marketing` | Narrativa | Claims sin evidencia, overselling, SEO/OG faltantes |

5.3 Aislamiento de red

Los tres entornos operan en redes Docker completamente separadas:

```
prod-net:   backend-prod + frontend-prod (puertos 3001/5173)
tests-net:  python-tests + backend-tests (puerto 3002)
audit-net:  6 agentes + coordinator (sin acceso a prod ni tests)
```

El coordinator espera la finalización de los seis agentes, agrega resultados por severidad (`high → medium → low`) y genera `REPORTE_CHESS_1.md` .

---

6. Experimento Canónico: Sesión Fundacional

6.1 Parámetros

| Parámetro | Valor |

|---|---|

| Semilla | 42 |

| Mapa | 60 × 60 celdas |

| Entidades iniciales | 8 (7 sociales + 1 gato) |

| Ticks ejecutados | 200 |

| Héroe fundador | Tryndamere |

| Refugio fundador | Refugio Fundador |

| Timestamp | 2026-03-08T12:53:48 |

6.2 Estado final de la población

| ID | Nombre | Tipo | Posición | Hambre | Energía |

|---|---|---|---|---|---|

| 1 | Ana | social | (47, 21) | 0.68 | 0.38 |

| 2 | Bruno | social | (0, 34) | **1.00** | 0.36 |

| 3 | Clara | social | (25, 5) | **1.00** | 0.26 |

| 4 | David | social | (10, 22) | **0.90** | **0.18** |

| 5 | Eva | social | (13, 5) | **1.00** | 0.40 |

| 6 | Félix | social | (0, 52) | **1.00** | 0.38 |

| 7 | Amiguisimo | gato | (12, 42) | **1.00** | 0.30 |

| 8 | Tryndamere | social | (2, 51) | **1.00** | 0.40 |

> *Salud = 1.0 para todos. Ningún agente alcanzó el estado de muerte.*

6.3 Métricas de acción

| Acción | Ejecuciones | Proporción |

|---|---|---|

| `SEGUIR` | 998 | 62.4% |

| `DESCANSAR` | 492 | 30.8% |

| `RECOGER_RECURSO` | 55 | 3.4% |

| `COMER` | 55 | 3.4% |

| **Total** | **1.600** | **100%** |

Veredicto del sistema: TENSIÓN

Alertas Watchdog: 65 (30 WARN + 35 CRITICAL)

6.4 Reproducibilidad completa

```
python cronica_fundacional.py --seed 42 --ticks 200 \
  --founder "Tryndamere" --refuge "Refugio Fundador"
```

El uso de `random.Random(entidad.id * 1000 + tick)` garantiza que cada ejecución produce exactamente el mismo conjunto de decisiones dado el mismo estado inicial.

---

7. Análisis: Comportamientos Emergentes

7.1 La trampa dinámica de la cohesión

El resultado más significativo es la **dominancia de `SEGUIR` (62.4%) en un contexto de escasez severa**. Este comportamiento no fue codificado como respuesta directa a la escasez. Emerge de la interacción entre tres mecanismos:

1. El modificador de relaciones sociales (confianza alta → bonus SEGUIR entre agentes con historia de cooperación)
2. La penalización de autonomía sobre exploración cuando `hambre > 0.85` (el agente con hambre crítica no puede explorar agresivamente)
3. La ausencia de recursos en las celdas cercanas (COMER y RECOGER no son viables la mayor parte del tiempo)

El resultado es una **trampa dinámica**: los agentes cohesionados en grupo mantienen relaciones de alta confianza que refuerzan el seguimiento mutuo, pero esto reduce la exploración individual necesaria para encontrar recursos. El grupo se desplaza unido hacia zonas que ya han sido agotadas por el propio grupo.

Este resultado es análogo al fenómeno de "información cascade" en economía del comportamiento (Bikhchandani et al., 1992): los agentes imitan el comportamiento del grupo incluso cuando esa imitación es colectivamente subóptima, porque la señal social (confianza, cohesión) supera la señal individual (hambre).

7.2 Agotamiento zonal y degradación espacial

Las 65 alertas se concentran en el cluster oeste del mapa (ticks 180–200). Los agentes en este cuadrante (Tryndamere, Félix, Amiguisimo) sufren hambre crítica crónica no por fallo del algoritmo de decisión sino por **agotamiento zonal**: han consumido los recursos locales disponibles y el modificador anti-oscilación (−0.40 sobre la celda anterior) dificulta el retroceso eficiente.

El modificador anti-oscilación fue diseñado para evitar el comportamiento de "ping-pong" (agente que oscila entre dos celdas sin progresar). Sin embargo, en condiciones de agotamiento zonal, este modificador tiene el efecto secundario no deseado de dificultar la migración de retroceso hacia zonas con recursos. Este trade-off es un ejemplo de **emergencia negativa**: una regla diseñada para mejorar el comportamiento promedio crea un caso extremo patológico.

7.3 El caso anómalo: Ana y la divergencia individual exitosa

Ana es la única entidad con hambre sub-crítica al final de la simulación (H=0.68), ubicada en (47, 21) — la posición más alejada de todos los demás agentes (distancia Euclidiana media ≈ 38 unidades al centroide del grupo).

Su aislamiento no fue una estrategia codificada ni una directiva del investigador. Fue el resultado acumulado de decisiones individuales de exploración que la llevaron progresivamente a una zona con densidad de recursos superior al cuadrante oeste donde se concentró el grupo.

Este resultado tiene una implicación teórica relevante: **la divergencia individual de la cohesión grupal puede ser una estrategia emergentemente superior bajo presión de escasez**, aunque el sistema no la codifica como tal. Un agente con rasgo `EXPLORADOR` tiene más probabilidad de reproducir el patrón de Ana por diseño; un agente `COOPERATIVO` tiene más probabilidad de quedar atrapado en la trampa de la cohesión.

7.4 Tabla resumen de fenómenos emergentes

| Fenómeno | Mecanismo generador | ¿Programado? |

|---|---|---|

| Cohesión bajo escasez | mod_relaciones > mod_autonomia en estadios medios | No (emergente) |

| Agotamiento zonal | mod_anti_oscilacion + densidad de recursos local | Parcial (efecto lateral) |

| Divergencia individual exitosa | rasgo_explorador + exploración no competida | No (emergente) |

| Supervivencia total | mod_autonomia activa en estados críticos | Sí (diseñado) |

| Fragmentación en 3 clusters | dispersión exploratoria + gradiente de recursos | Parcial (emergente) |

---

8. Trazabilidad Radical como Propiedad de Diseño

8.1 Definición operacional

La trazabilidad radical, tal como se implementa en Artificial World, implica que para cualquier decisión del sistema en cualquier tick, existe una cadena completa de evidencia verificable:

```
Tick T → Estado(agente_i, T) → Candidatos(T) → Scores(T) → Decisión(T) → Efecto(T)
```

Y para cualquier claim sobre el sistema:

```
Claim → Evidencia(código || test || log) → Verificable_sin_acceso_al_autor
```

8.2 La taxonomía real/demo/roadmap

Una de las contribuciones metodológicas más importantes de este proyecto es la taxonomía explícita del estado de cada componente:

| Estado | Definición | Ejemplos en el proyecto |

|---|---|---|

| **REAL** | Existe, está testeado, es reproducible | Motor Python, persistencia, Modo Sombra, 11 suites |

| **DEMO** | Existe, funciona, pero no es el motor principal | Web fullstack, DobackSoft en este repo |

| **PARCIAL** | Existe pero con funcionalidad incompleta | HeroRefuge, CivilizationSeed web, IA local |

| **ROADMAP** | Diseñado pero no implementado | 3D runtime, integración Python/JS, diplomacia |

Esta taxonomía es pública, está documentada en el repositorio y es verificable por cualquier persona que lea el código. No es una concesión a la transparencia: es una herramienta activa de gestión de producto.

8.3 El Sistema Chess como implementación institucional

El Sistema Chess institucionaliza la trazabilidad radical al nivel de proceso: los agentes auditores no pueden ser influenciados por el equipo que construye, no tienen acceso de escritura y no tienen estado entre ejecuciones. Cada reporte es verificable, reproducible y fechado.

Este diseño refleja una convicción filosófica documentada en el Manifiesto del proyecto:

> *"Una respuesta sin rastro no es una base seria de decisión."*

---

9. Arquitectura Conceptual: El Modelo de Civilizaciones Vivas

9.1 Entidades de dominio y relaciones

El modelo conceptual que subyace a ambas capas técnicas define ocho entidades de primer orden:

```
CivilizationSeed
    │ condiciona tono y valores
    ▼
  World ─── tiene ──► Refuge (fundador)
    │                     │
    │                  organiza
    │                     │
    │                     ▼
    │                  Community
    │                     │
    │                  incluye / lidera
    │                     │
    └──── genera ──────► Hero
                          │
                       produce
                          │
                          ▼
                    Event → MemoryEntry → HistoricalRecord
                                                │
                                            alimenta
                                                │
                                                ▼
                                            Territory / Route
                                      (lectura estratégica 2D)
```

9.2 El flujo fundador mínimo

El flujo mínimo verificable para crear una civilización es:

```
1. Usuario elige CivilizationSeed (uno de 7 arquetipos)
2. Usuario nombra al héroe fundador
3. Usuario nombra el refugio fundador
4. Sistema crea:
   - Hero (fundador, con rol "Constructor de Mundos")
   - Refuge (recursos: food=60, shelter=75, security=55, morale=58)
```

```
    - Community (cultura y tensiones derivadas de la semilla)
    - MemoryEntry ("fundación iniciada")
    - World (contenedor global con tick=0)
5. Simulación avanza produciendo eventos → memoria → historia
```

Tiempo total: < 2 minutos desde onboarding hasta mundo activo.

9.3 La regla 2D/3D: separación de verdad y presencia

Una decisión arquitectónica fundamental del sistema es la separación estricta entre capas:

| Capa | Rol | Estado actual |

|---|---|---|

| **2D** | Verdad sistémica: mapa, grid, rutas, nodos, recursos, refugios, influencia, fronteras | Implementada |

| **3D** | Encarnación futura: héroes de gran presencia, refugios encarnados, eventos históricos visuales | **No implementada (Roadmap)** |

Esta separación evita la confusión frecuente en simulaciones de vida artificial entre la lógica del sistema (donde vive la verdad) y su representación visual. La capa 3D, cuando se implemente, será subordinada a la verdad sistémica 2D, no un sistema paralelo con su propia lógica.

---

10. Limitaciones y Trabajo Futuro

10.1 Limitaciones actuales verificadas

L1. Ausencia de integración Python/JS. Los dos motores son sistemas independientes sin comunicación en tiempo real. La verdad sistémica de alta fidelidad (motor Python) y la demo web (motor JS) no comparten estado. Esta es una limitación conocida y documentada, no un error no detectado.

L2. Escasez de eventos narrativos intermedios. La sesión de 200 ticks registró solo 55 eventos de intercambio de recursos (COMER, RECOGER). Los eventos de `compartio`, `robo` y `ataque`, aunque implementados, fueron marginales porque sus condiciones de viabilidad son más restrictivas que SEGUIR y DESCANSAR.

L3. Comunidad como contrato de datos, no como simulación social. El concepto de `Community` existe con estado inicial (cultura, tensiones, cohesión, liderazgo) pero no como simulación social rica. Las relaciones inter-agente son diádicas; no hay dinámicas grupales emergentes de normas o liderazgo.

L4. Diplomacia y linajes no implementados. Civilizaciones completas con diplomacia, guerras, migraciones y linajes permanecen en roadmap.

L5. Historial multi-sesión no agregado. Cada ejecución produce su propia crónica pero el sistema aún no compara ni agrega historial entre sesiones distintas con la misma semilla.

L6. Capturas visuales no integradas en crónica. El sistema genera la crónica en JSON y Markdown pero no incluye screenshots automáticos del estado del mundo en ticks clave.

10.2 Líneas de trabajo futuro

F1. Mecanismo de migración adaptativa. Implementar un subsistema que detecte el agotamiento zonal y genere una directiva grupal de migración cuando la densidad de recursos en el cuadrante actual cae por debajo de un umbral. Esto transformaría la trampa dinámica identificada en §7.1 en comportamiento adaptativo verificable.

F2. Historia profunda: más eventos → más crónica. Hacer que acciones como `compartio`, `robo`, `atacó`, `huyó_de` alimenten directamente `MemoryEntry` e `HistoricalRecord`, generando crónicas más ricas que reflejen la dinámica real de la simulación.

F3. Comunidades con normas emergentes. Implementar un mecanismo donde la frecuencia de ciertos tipos de acción en un grupo (alta frecuencia de COMPARTIR, baja frecuencia de ROBAR) se traduzca en una norma cultural que modifique los modificadores de utilidad de todos los miembros del grupo.

F4. Puente Python/JS. Definir un protocolo de API REST o WebSocket que permita que el motor Python sea el backend de verdad sistémica y el motor JS sea la capa de presentación web en tiempo real. El roadmap técnico en 4 fases está documentado en `docs/ARTIFICIAL_WORD_CRONOGRAMA.md`.

F5. Integración del Sistema Chess en CI. Ejecutar el ciclo de auditoría Chess automáticamente en cada push, no solo manualmente, generando un reporte diferencial que identifique regresos en la calidad del código.

F6. CivilizationSeeds condicionando efectivamente el runtime. Los siete arquetipos existen como estado fundacional en la capa web, pero no condicionan aún el comportamiento de los agentes en el motor Python. La integración de la semilla en la distribución inicial de rasgos de agente es el siguiente paso lógico.

---

11. Discusión: La Filosofía de Sistemas que se Explican Solos

11.1 Por qué la trazabilidad es una ventaja competitiva

En el contexto actual de proliferación de sistemas de IA con capacidades difíciles de verificar externamente, la trazabilidad radical no es solo un valor ético: es una **ventaja competitiva demostrable**. Un sistema que puede decir "esta decisión ocurrió en el tick 189, fue tomada por el agente Tryndamere, la acción ganadora fue SEGUIR con puntuación 0.73, y los cinco modificadores que más la elevaron fueron X, Y, Z, W, V" es cualitativamente diferente de un sistema que devuelve un comportamiento sin rastro.

Este principio, central en el diseño de Artificial World, tiene implicaciones más amplás para el diseño de sistemas de IA en general: **la opacidad de un sistema no es un defecto técnico inevitable sino una elección de diseño**.

11.2 El modelo de honestidad técnica como metodología

La taxonomía real/demo/roadmap que Artificial World implementa activamente es, en sí misma, una metodología de gestión de producto y comunicación técnica. El criterio de éxito documentado en múltiples archivos del proyecto ("una persona nueva entiende en menos de 5 minutos qué es real y qué es demo") es un indicador de producto, no solo de documentación.

Esta metodología tiene un costo: requiere resistir la tentación de presentar el sistema en su estado óptimo teórico. Pero el beneficio es la **credibilidad acumulada**: cada claim verificable fortalece la confianza en los claims no verificados de inmediato.

11.3 Local, auditable, open source: una tesis sobre dónde vive la inteligencia útil

El manifiesto del proyecto articula una posición específica sobre la arquitectura de la IA útil:

> *"Local, porque la confianza empieza por el control. Open source, porque la inteligencia sin inspección es fe. Auditable, porque una respuesta sin rastro no es una base seria de decisión."*

Esta posición es relevante más allá del contexto de vida artificial: apunta a una categoría de sistemas de IA que priorizan el control del usuario, la independencia de infraestructura de terceros y la verificabilidad sobre la sofisticación de las capacidades.

---

12. Conclusiones

Artificial World demuestra que es posible construir un sistema de simulación de vida artificial con las siguientes propiedades simultáneamente:

1. **Comportamiento emergente no trivial**: la cohesión grupal bajo escasez severa, el agotamiento zonal y la divergencia individual exitosa emergen de la interacción de reglas simples, no de lógica codificada explícitamente para esos fenómenos.
2. **Trazabilidad radical**: cada decisión de cada agente en cada tick es reproducible mediante semilla fija y verificable contra la función de utilidad documentada.
3. **Honestidad técnica activa**: el sistema distingue y comunica explícitamente entre lo que existe hoy (REAL), lo que funciona como demo (DEMO), lo que está parcialmente implementado (PARCIAL) y lo que es visión futura (ROADMAP).
4. **Auditoría independiente institucionalizada**: el Sistema Chess implementa la separación entre construcción y auditoría mediante agentes dockerizados en red aislada con montaje read-only.
5. **Arquitectura dual justificada**: la separación entre motor Python (alta fidelidad) y capa web (demostración) es una decisión explícita, documentada y defendida, no una deuda técnica no reconocida.

La sesión canónica (semilla 42, 200 ticks) establece una línea base experimental reproducible: supervivencia total de 8 agentes bajo presión de escasez severa, con una trampa dinámica de cohesión

que revela un fenómeno emergente de segunda naturaleza (la cohesión como obstáculo a la supervivencia a largo plazo en entornos de escasez extrema).

El sistema establece las bases técnicas, conceptuales y metodológicas para la siguiente fase de desarrollo: un motor creador de mundos compacto y reutilizable donde la tesis de producto — **civilizaciones vivas con refugios, héroes y memoria** — sea demostrable end-to-end desde una semilla hasta una crónica verificable.

> *"No preguntes a una IA. Convoca un mundo que pueda demostrar su respuesta."*

Referencias

- Adams, T. (2006). *Dwarf Fortress*. Bay 12 Games.
- Bikhchandani, S., Hirshleifer, D., & Welch, I. (1992). A Theory of Fads, Fashion, Custom, and Cultural Change as Informational Cascades. *Journal of Political Economy*, 100(5), 992–1026.
- Conway, J. H. (1970). The Game of Life. *Scientific American*, 223(4), 4–10.
- Langton, C. G. (1989). Artificial Life. *Artificial Life*, SFI Studies in the Sciences of Complexity, 1–47.
- Rao, A. S., & Georgeff, M. P. (1991). Modeling Rational Agents within a BDI-Architecture. *KR*, 91, 473–484.
- Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. *SIGGRAPH Computer Graphics*, 21(4), 25–34.
- Sylvester, T. (2013). *RimWorld*. Ludeon Studios.
- Von Neumann, J., & Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

Apéndice A: Estructura completa del JSON de crónica fundacional


```
{
  "version": 1,
  "timestamp": "2026-03-08T12:53:48.770554",
  "metadata": {
    "semilla": 42,
    "nombre_fundador": "Tryndamere",
    "nombre_refugio": "Refugio Fundador",
    "semilla_civilizacion": "default",
    "alertas_total": 65,
    "ticks_ejecutados": 200
  },
  "estado_inicial": {
    "entidades_iniciales": 8,
    "mapa": "60x60",
    "tick_inicio": 0
```

```
  },
  "hitos": [
    { "tick": 0,   "tipo": "inicio",        "descripcion": "Fundación iniciada. Comunidad des
    { "tick": 50,  "tipo": "primer_ciclo",   "descripcion": "Primeras interacciones sociales
    { "tick": 100, "tipo": "mitad",          "descripcion": "Hambre generalizada. Primeras al
    { "tick": 199, "tipo": "crisis",         "descripcion": "65 alertas. 7/8 entidades en ham
    { "tick": 200, "tipo": "cierre",         "descripcion": "Sesión completada. Supervivencia
  ],
  "metricas": {
    "siguio": 998,
    "descanso": 492,
    "recogio_recurso": 55,
    "comio": 55,
    "total_acciones": 1600
  },
  "veredicto": "tension",
  "resumen": "Supervivencia en tensión. 8 entidades vivas. Hambre máxima 1.00."
}
```


Apéndice B: Reproducción de la sesión canónica

```
# Instalar dependencias
pip install -r requirements.txt

# Ejecutar sesión reproducible (headless)
$env:SDL_VIDEODRIVER="dummy"
$env:SDL_AUDIODRIVER="dummy"
python cronica_fundacional.py --seed 42 --ticks 200 `
  --founder "Tryndamere" --refuge "Refugio Fundador"

# Artefactos generados:
# cronica_fundacional.json  ← datos completos verificables
# cronica_fundacional.md    ← resumen legible
# cronica_fundacional.pdf   ← documento LaTeX compilado (requiere MiKTeX)

# Ejecutar suite completa de tests
python pruebas/run_tests_produccion.py
```

Apéndice C: Taxonomía de componentes — Estado verificado

| Componente | Estado | Evidencia verificable |

|---|---|---|

| Motor Python 2D | REAL | `principal.py`, `nucleo/simulacion.py`, 13 acciones |

| Función de utilidad | REAL | `agentes/motor_decision.py` (415 líneas) |

| Memoria espacial/social | REAL | `systems/memory/`, `agentes/relaciones.py` |

| Persistencia SQLite | REAL | `mundo_artificial.db`, `sistema_persistencia.py` |

| Modo Sombra | REAL | `gestor_modo_sombra.py`, suite de tests |

| Watchdog | REAL | `SistemaWatchdog`, 65 alertas en sesión canónica |

| Crónica fundacional | REAL | JSON + MD + PDF desde runtime |

| 11 suites Python (68+ tests) | REAL | `pruebas/run_tests_produccion.py` |

| CI GitHub Actions | REAL | `.github/workflows/ci-completo.yml` |

| Web fullstack (Node+React) | DEMO | `backend/src/`, `frontend/src/` |

| HeroRefuge | PARCIAL | `heroRefuge.js`, hasta 256 mundos persistentes |

| CivilizationSeed (7 arquetipos) | PARCIAL | `civilizationSeeds.js`, estado fundacional completo |

| IA local (Ollama) | PARCIAL | `aiCore.js`, fallback determinista |

| Sistema Chess | REAL | `docker/agents/`, `docker/coordinator/` |

| Runtime 3D | ROADMAP | Diseñado, no implementado |

| Integración Python/JS | ROADMAP | Documentado, no implementado |

| Diplomacia / linajes | ROADMAP | Conceptualizado, no implementado |

Artificial World — Constrúyelo. Habítalo. Haz que crezca.

Motor creador de mundos — El destino.

Artificial World · Cosigein SL · 2026 · Constrúyelo. Habítalo. Haz que crezca.